



Assignment 1

Lab Course: Object Oriented Programming

Course code: CSC241

Submitted by:

Mubashira Allah Dad

SP26-BCS-031

Submitted to:

Dr Muhammad Shahid Bhatti

Smart Campus Learning Space Allocation System

Task 1: Stakeholder and Requirement Analysis

The stakeholders and their core needs can be:

Stakeholders	Main Needs
Students	Students need fair access to suitable spaces for individual and group activities.
Faculty Members	Faculty members need priority access for scheduled teaching and urgent academic activities.
Administrators	Efficient room utilization and management of reservations to justify space planning and investment.
Security Staff	Security staff need traceable access records while protecting users' private information.
Maintenance Staff	The authority to take an unsafe room out of service , even if it is already booked.

Requirement Conflicts:

1. Fairness vs Faculty Priority:

Problem:

Students expect fair access to rooms, while faculty may need priority for teaching and urgent academic activities.

Decision needed:

We can assume that scheduled teaching activities receive priority, while non-teaching requests are handled using a fair allocation rule.

Fact:

Different types of users may compete for the same limited learning spaces.

Assumption:

The university will establish rules that determine which requests receive priority when there is a conflict.

2. Room Utilization vs Safety:

Problem:

Administration wants rooms to be used efficiently, but maintenance staff must be able to immediately block an unsafe room.

Decision needed:

We can assume that safety takes priority over utilization. Once maintenance marks a room unsafe, the system prevents new reservations and handles existing reservations according to the emergency/maintenance policy.

Fact:

Maintenance staff must be able to block unsafe rooms immediately.

Assumption:

Safety and maintenance restrictions take priority over normal room booking requests.

3. Privacy vs Security Traceability:

Problem:

Security staff need access records that can be traced, but ordinary users should not see another user's identity or reservation purpose.

Decision needed:

We can assume that the system stores detailed access/reservation information for authorized staff, while ordinary users only see room availability, not the person's identity or reservation purpose.

Fact:

The system must support traceable access records while protecting users' privacy.

Assumption:

Detailed reservation and access information will only be visible to authorized personnel.

Task 2: Object Oriented Conceptual Model

Classes:

Classes	Main Responsibility	Attributes/Operations
User	A person who interacts with or depends on the system and manages their information and roles.	User ID, Name, Email requestSpace() viewAvailability()

Role	Represents a user's role and permissions.	roleName, permissions addPermission() removePermission()
Room	Manage information about a learning space and its availability.	roomID, capacity, roomType checkAvailability() setUnavailable()
Reservation	Manages an area's reservation and its state.	reservationID, dateTime, purpose createReservation() cancelReservation()
Approval	Handles the approval decision for reservation requests according to the applicable rules.	approvalID, approvalStatus, approvalDate approveRequest() rejectRequest()
Equipment	Manages equipment required for a particular activity or room reservation.	equipmentID, equipmentType checkAvailability() allocateEquipment()
Maintenance Block	Represents a period during which a room is unavailable because of maintenance or safety concerns.	blockID, reason, startTime, endTime blockRoom() removeBlock()
AccessRecord	Keeps a record of authorized access to campus spaces for security and auditing.	recordID ,accessTime accessType createRecord() viewRecord()
Notification	Send messages about important reservation events	notificationID , message sendNotification() markAsRead()

UML Class Relationships:



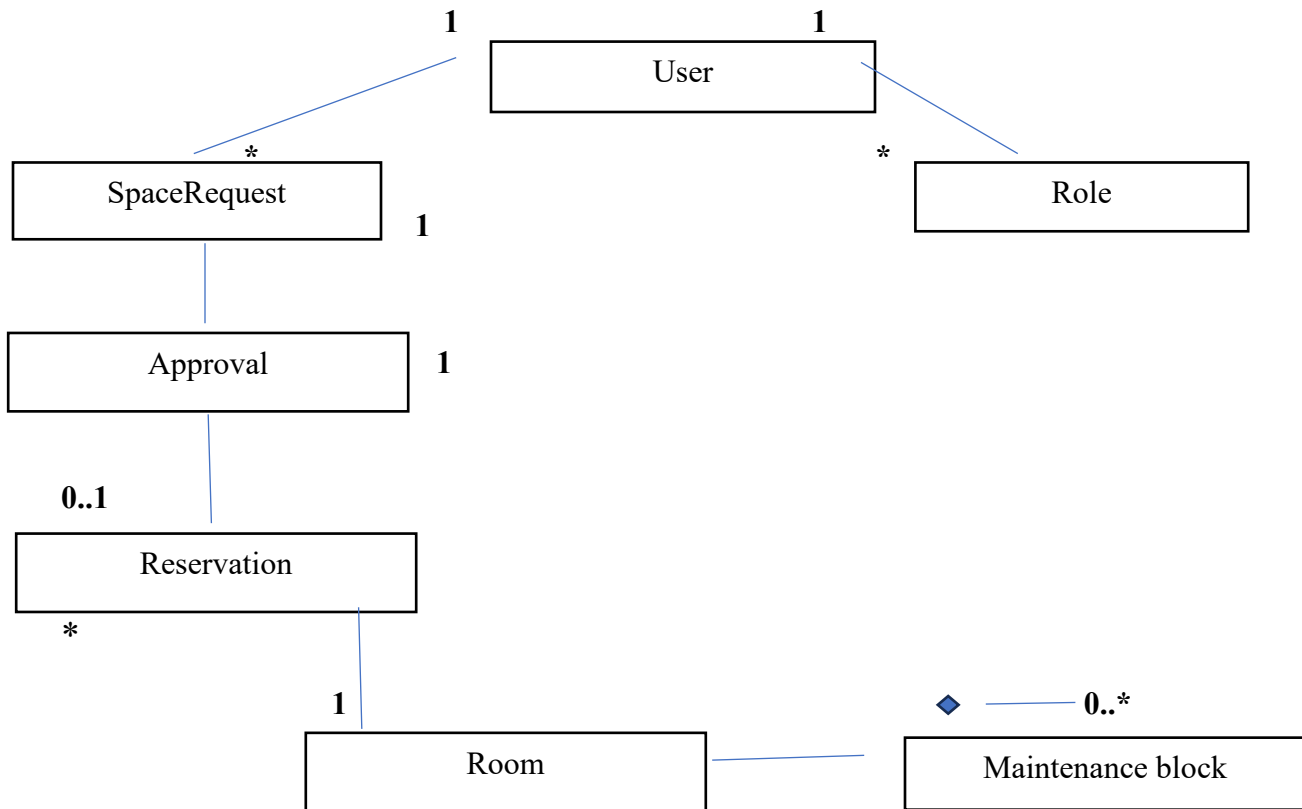


Figure 1: UML Class Diagram of the Smart Campus Learning Space Allocation System. The diagram separates user requests, approvals, reservations, rooms, maintenance block, notifications, roles, and access records into distinct classes. Associations show how objects interact, while the composition relationship between Room and Equipment represents equipment managed as part of a room's resources. Privacy is handled through User/Role access control and restricted AuditRecord information.

Relationships and Multiplicities:

- One User can create many SpaceRequests: 1 : *.
- One User can have multiple Role objects.
- One SpaceRequest has one Approval.
- A SpaceRequest may result in zero or one Reservation: 1 : 0..1.

- Many Reservations can be associated with one Room at different times: * : 1.
- A Room contains zero or more Equipment objects: 1 : 0..*.
- A Reservation can generate notifications.
- A User can have many AuditRecords.

Composition and Responsibility Separation:

Each class has a separate responsibility. User manages users and roles, SpaceRequest handles requests, Approval handles approvals, Reservation manages bookings, Room manages space availability, MaintenanceBlock handles maintenance periods, Notification sends updates, and AccessRecord maintains audit information.

The User–Role design supports multiple and changing roles. It also helps enforce Privacy First by restricting sensitive information to authorized users. This separation prevents one class from controlling the whole system and makes the design easier to maintain.

Task 3: Abstraction, Hidden Information and Encapsulation

Three important classes are User, Room and Reservation.

1. User

The **User** class represents a person who interacts with or uses the Smart Campus Learning Space Allocation System. It provides only the information and actions needed by other parts of the system.

Hidden information:

- User's authentication details
- Private contact information
- Personal information related to permissions

Controlled operations:

- requestSpace()
- viewReservations()
- receiveNotification()

Sensitive user and reservation information is not directly accessible to other users. Access is controlled through the user's roles and permissions, supporting the **Privacy First** requirement.

2. Room

The Room class represents a learning space and manages its availability and status.

Hidden information:

- Current availability
- Maintenance status
- Internal room-management details

Controlled operations:

- isAvailable()
- markUnavailable()
- checkAccessibility()

Other classes cannot directly change the room's availability. Controlled methods ensure that a room cannot be incorrectly marked available or used when it is blocked for maintenance or safety.

3. Reservation

The **Reservation** class represents a confirmed or requested booking of a learning space and manages its lifecycle.

Hidden information:

- Reservation status
- Booking time and date
- Reservation details and purpose

Controlled operations:

- confirm () relocate () cancel ()
- getStatus () getStatus ()

The reservation status should not be changed directly by other classes. Controlled operations ensure that reservations follow valid rules and that affected users can be notified when a reservation is cancelled or relocated.

Invariants:

The following conditions must always remain true:

1. A room cannot have two confirmed reservations for overlapping times.

2. A room blocked for maintenance or safety cannot have a new confirmed reservation during that period.
3. Every reservation must be linked to one valid user and one valid room.
4. A cancelled reservation cannot be directly changed back to confirmed without going through the proper reservation process.
5. Ordinary users cannot view another user's identity or reservation purpose.

Encapsulation:

Encapsulation means keeping important attributes private and allowing access through controlled methods. For example, other classes should not directly change a room's availability or a reservation's status. Instead, they use methods such as `markUnavailable()`, `confirm()`, or `cancel()`. The class can then check the rules before making a change. This prevents invalid states and helps maintain the system's invariants. Privacy is also protected by restricting sensitive users and reservation information to authorized roles.

Task 4: Alternative Designs and Decision

Inheritance:

In an inheritance-based design, separate subclasses such as Student, Faculty, Administrator, and SecurityStaff would inherit from a common User class. Each subclass could contain behavior specific to that role. However, this design becomes less flexible when a person has multiple roles or when their permissions change.

Composition / Role Objects:

In a composition-based design, there is one User class that can be connected to one or more Role objects, such as Student, Faculty, or Administrator. Each Role contains its permissions. This allows a user to have multiple roles and allows permission to change without creating new subclasses.

Comparison of Alternative Designs:

Criteria	Inheritance	Composition/Role Objects
Extensibility	Adding new roles may require new subclasses.	New roles can be added as Role objects.
Multiple Roles	Difficult because one user may need several subclasses.	Easy because one User can have multiple Role objects.

Changing Permissions	May require changes to classes.	Permissions can be changed through the Role object.
----------------------	---------------------------------	---

Task 5: Behavior and Edge Cases

Reservation Life Cycle:

The reservation follows these main states:

Requested → Pending → Confirmed → Completed

A reservation may also move to Cancelled if it is rejected or cancelled. During an emergency, a confirmed reservation may move to Relocated and then become Confirmed again after a suitable alternative room is assigned.

Edge Cases

1. Simultaneous Requests:

If two users request the same room for overlapping times, the system checks availability and confirms only a valid reservation. This prevents overlapping confirmed bookings.

2. Maintenance After Confirmation:

If a confirmed room becomes unavailable because of maintenance or safety issues, the Reservation is cancelled or relocated, and the affected user is notified.

3. Emergency Cancellation/Relocation:

If a building is closed during an emergency, affected reservations are identified and cancelled or relocated to available spaces. Notifications are sent to affected users.

Responsibility:

The **Room** class checks whether a space is available, while the **Reservation** class controls the reservation status and lifecycle. The **Notification** class informs affected users about cancellations or relocations. This separation ensures that each class handles its own responsibility.

Task 6 — Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) gives users permissions according to their assigned roles instead of giving permissions to each user separately. A user can have one or more roles, and each role contains specific permissions. This makes access control easier to manage when users have different responsibilities.

In the Smart Campus Learning Space Allocation System, RBAC can be applied through the **User** and **Role** classes. For example, a student can submit a space request, an administrator can approve requests, maintenance staff can block a room, and security staff can access audit records. The system checks the user's role before allowing a particular operation. RBAC is especially useful for the **Privacy First** requirement. Ordinary users can view room availability but cannot view another user's identity or reservation purpose. Authorized roles can access sensitive information when required for security or administration. This approach also supports changing and multiple roles without creating separate subclasses for every user type.

References

Keep the two references already in your draft:

1. Ferraiolo, D. F., Cugini, J., & Kuhn, D. R. (1995). *Role-Based Access Control (RBAC): Features and Motivations*. NIST.
2. Sandhu, R., Ferraiolo, D., & Kuhn, D. (2000). *The NIST Model for Role-Based Access Control: Towards a Unified Standard*. IEEE Computer.
3. Jansen, W. (1998). *A Revised Model for Role-Based Access Control*. NIST Interagency Report 6192. National Institute of Standards and Technology.